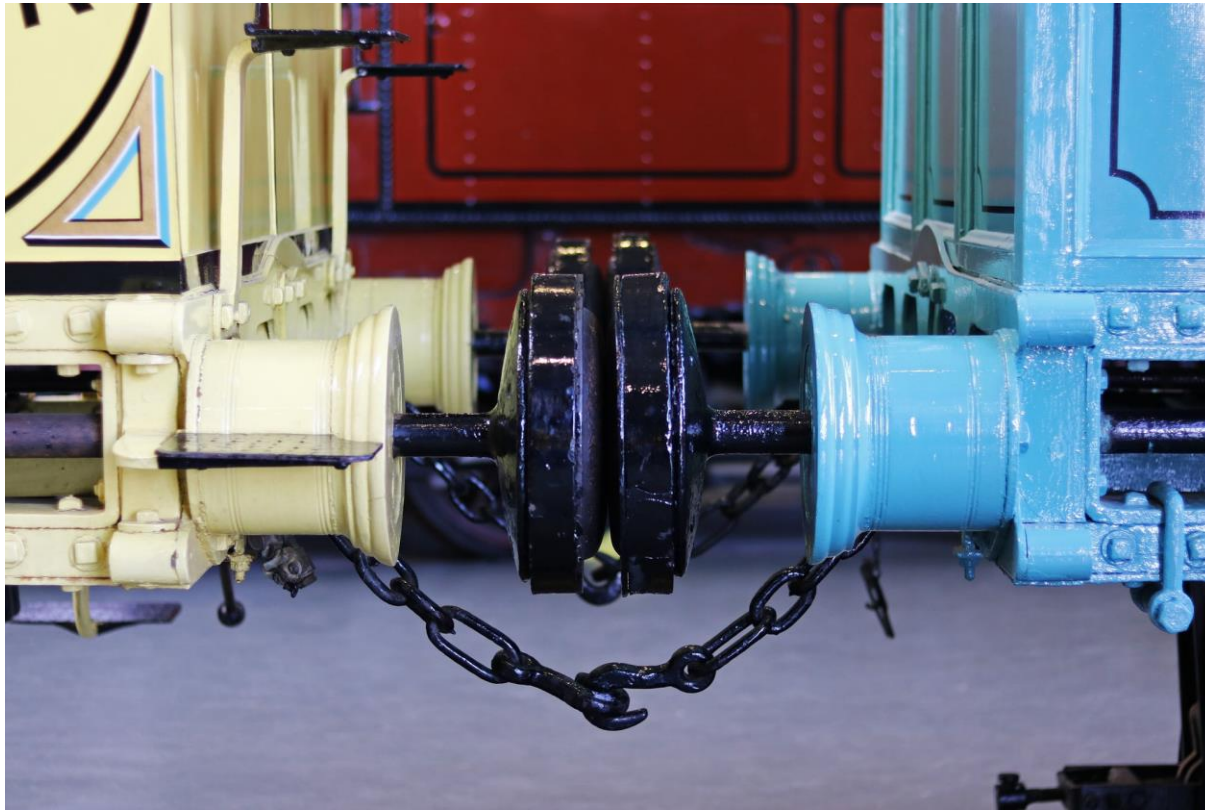


UNIT III: 3.3 Joins – INNER and OUTER JOINS, Sub Queries



Prepared By

Mohammad Ali

Lecturer (Selection Grade)

M. H. Saboo Siddik Polytechnic, Mumbai

Objectives

After completing this lesson, you should be able to do the following:

- Horizontally combine data from multiple tables.
- Distinguish between inner and outer SQL joins.
- Write `SELECT` statements to access data from more than one table using equijoins and non-equijoins.
- Join a table to itself by using a self-join

Combining Data from Multiple Tables

SQL uses set operators to combine tables vertically.



SQL uses joins to combine tables horizontally.



Poll 

Quiz

01. Multiple Choice Poll

Which of these statements is used to combine tables horizontally?

- a. SET
- b. APPEND
- c. JOIN
- d. INPUT
- e. CONCAT

01. Multiple Choice Poll – Correct Answer

Which of these statements is used to combine tables horizontally?

- a. SET
- b. APPEND
- ☒ c. JOIN
- d. INPUT
- e. CONCAT

Types of Joins

PROC SQL supports two types of joins:

- Inner joins – equijoin , Natural joins
- Outer joins – Left Outer Join, Right Outer join, Full Outer Join
- Cross joins / Cartesian Product
- USING clause

Joining Tables Using SQL

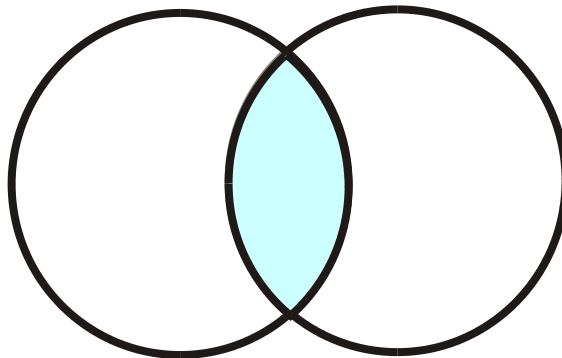
SYNTAX

```
SELECT  table1.column, table2.column
FROM    table1
[NATURAL JOIN table2] |
[JOIN table2 USING (column_name)] |
[JOIN table2
    ON (table1.column_name = table2.column_name)] |
[LEFT|RIGHT|FULL OUTER JOIN table2
    ON (table1.column_name = table2.column_name)] |
[CROSS JOIN table2];
```


Types of Joins

Inner joins

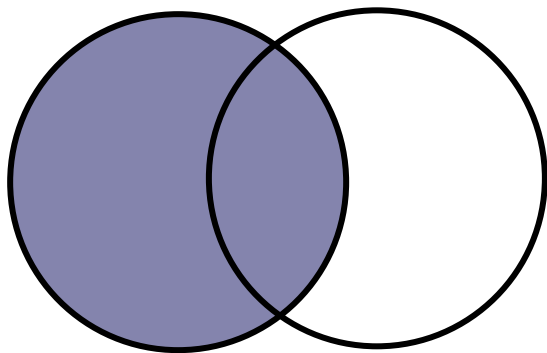
- return only matching rows
- enable a maximum of 256 tables to be joined at the same time.



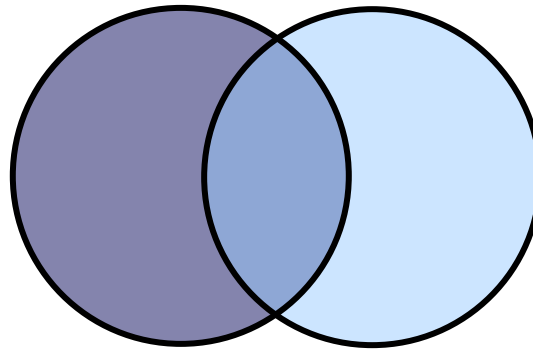
Types of Joins

Outer joins

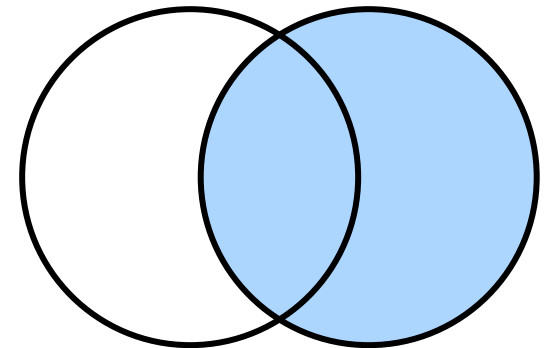
- return all matching rows, plus nonmatching rows from one or both tables
- can be performed on only two tables or views at a time.



Left



Full



Right

Cartesian Product

- To understand how SQL processes a join, it is important to understand the concept of the Cartesian product.
- A query that lists multiple tables in the FROM clause without a WHERE clause produces all possible combinations of rows from all tables.
- This result is called the ***Cartesian product***.
- The **CROSS JOIN** clause produces the cross-product of two tables.
- This is also called a Cartesian product between the two tables.

```
SELECT <columns>  
FROM <table-name1>, <table-name2>;
```

OR

```
SELECT <columns>  
FROM <table-name1>  
CROSS JOIN <table-name2>;
```

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

```
SELECT *  
FROM One, Two;
```

OR

```
SELECT *  
FROM One  
CROSS JOIN Two;
```

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v



Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

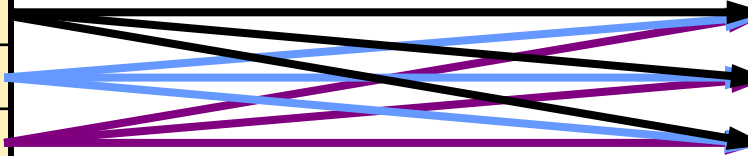
Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

Table Two

ID	ADD
2	x
3	y
5	v



Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

3 rows

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

3 rows

Table Two

ID	ADD
2	x
3	y
5	v

3 rows

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

Cartesian Product

Table One

ID	NAME
1	a
4	d
2	b

3 rows X 3 rows

Table Two

ID	ADD
2	x
3	y
5	v

Result Set

ID	NAME	ID	ADD
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

9 rows

Cartesian Product

- A Cartesian product is formed when:
 - A join condition is omitted
 - A join condition is invalid
 - All rows in the first table are joined to all rows in the second table
- To avoid a Cartesian product, always include a valid join condition.
- A Cartesian product is rarely the **desired** result of a query.
- The number of rows in a Cartesian product is the product of the number of rows in the contributing tables.

$$3 \times 3 = 9$$

$$1,000 \times 1,000 = 1,000,000$$

$$100,000 \times 100,000 = 10,000,000,000$$

Poll

Quiz



02. Quiz

How many rows are returned from this query?

```
select *  
  from three, four;
```

Table Three

X	A
1	a1
1	a2
2	b1
2	b2
4	d

Table Four

X	B
2	x1
2	x2
3	y
5	v

02. Quiz – Correct Answer

How many rows are returned from this query?

The query produces 20 rows.

```
select *  
  from three, four;
```

Table Three

ID	NAME
1	a1
1	a2
2	b1
2	b2
4	d

Table Four

ID	ADD
2	x1
2	x2
3	y
5	v

$$5 \times 4 = 20$$

Partial Results Set

ID	NAME	ID	ADD
1	a1	2	x1
1	a1	2	x2
1	a1	3	y
1	a1	5	v
1	a2	2	x1
1	a2	2	x2
1	a2	3	y
1	a2	5	v
2	b1	2	x1
2	b1	2	x2

Inner Joins

- We use an SQL **INNER JOIN** when we need to match rows from two tables.
- Rows that match remain in the result, those that don't are rejected.
- The match condition is commonly called the join condition.
- When the match conditions involve equality, that is matching exactly the contents of one column to another, the sql inner join is called an **Equijoin**.
- **Natural Join** is a type of Inner join which is based on column having same name and same datatype present in both the tables to be joined.
- A common situation is where we need to join the primary key of one table to the foreign key of another.

Natural Joins

- **Natural Join** is a type of Inner join which is based on column having same name and same datatype present in both the tables to be joined.
 - It selects rows from the two tables that have equal values in all matched columns.
 - If the columns having the same names have different data types, an error is returned.
- A common situation is where we need to join the primary key of one table to the foreign key of another.

Inner Joins

Inner join syntax resembles Cartesian product syntax, but a WHERE clause restricts which rows are returned.

General form of an inner join:

```
SELECT column-1<, ...column-n>  
  FROM table-1|view-1<, ... table-n|view-n>  
  WHERE join-condition(s)  
        <AND other subsetting conditions>  
        <other clauses>;
```

The syntax for Natural Join is:

```
SELECT * FROM table-name1  
NATURAL JOIN table-name2
```


Inner Joins

Inner join syntax resembles Cartesian product syntax, but a WHERE clause restricts which rows are returned.

General form of an inner join:

```
SELECT column-1<, ...column-n>  
FROM table-1|view-1<, ... table-n|view-n>  
WHERE join-condition(s)  
      <AND other subsetting conditions>  
      <other clauses>;
```

Significant syntax changes from earlier queries:

- The FROM clause references multiple tables.
- The WHERE clause includes join conditions in addition to other subsetting specifications.

Natural Joins

```
SELECT * FROM emp NATURAL JOIN dept ;
```

```
SQL> select * from emp natural join dept;
```

DEPTNO	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DNAME	LOC
=====	=====	=====	=====	=====	=====	=====	=====	=====	=====
10	7782	CLARK	MANAGER	7839	09-JUN-81	2450		ACCOUNTING	NEW YORK
10	7839	KING	PRESIDENT		17-NOV-81	5000		ACCOUNTING	NEW YORK
10	7934	MILLER	CLERK	7782	23-JAN-82	1300		ACCOUNTING	NEW YORK
20	7566	JONES	MANAGER	7839	02-APR-81	2975		RESEARCH	DALLAS
20	7902	FORD	ANALYST	7566	03-DEC-81	3000		RESEARCH	DALLAS
20	7876	ADAMS	CLERK	7788	23-MAY-87	1100		RESEARCH	DALLAS
20	7369	SMITH	CLERK	7902	17-DEC-80	800		RESEARCH	DALLAS
20	7788	SCOTT	ANALYST	7566	19-APR-87	3000		RESEARCH	DALLAS
30	7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	SALES	CHICAGO
30	7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	SALES	CHICAGO
30	7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	SALES	CHICAGO
30	7900	JAMES	CLERK	7698	03-DEC-81	950		SALES	CHICAGO
30	7698	BLAKE	MANAGER	7839	01-MAY-81	2850		SALES	CHICAGO
30	7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	SALES	CHICAGO

```
14 rows selected.
```

Inner Joins

Conceptually, when processing an inner join, SQL does the following:

1. builds the Cartesian product of all the tables listed
2. applies the WHERE clause to limit the rows returned

Inner Joins: Cartesian Product Built

Table One

X	A
1	a
4	d
2	b

Table Two

X	B
2	x
3	y
5	v

```
select *  
  from one, two
```

X	A	X	B
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

Inner Joins: WHERE Clause Restricts Rows

Table One

X	A
1	a
4	d
2	b

Table Two

X	B
2	x
3	y
5	v

```
select *  
  from one, two  
 where one.x=two.x;
```

X	A	X	B
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v



Inner Joins: Results Are Returned

Table One

X	A
1	a
4	d
2	b

Table Two

X	B
2	x
3	y
5	v

```
select *  
  from one, two  
 where one.x=two.x;
```

X	A	X	B
2	b	2	x



Tables do not have to be sorted before they are joined.

Inner Joins

One method of displaying the X column only once is to use a table qualifier in the SELECT list.

Table One

X	A
1	a
4	d
2	b

Table Two

X	B
2	x
3	y
5	v

```
select one.x, a, b  
  from one, two  
 where one.x=two.x;
```

X	A	B
2	b	x

Inner Joins

Display all combinations of rows with matching keys, including duplicates.

Table Three

X	A
1	a1
1	a2
2	b1
2	b2
4	d

Table Four

X	B
2	x1
2	x2
3	y
5	v

```
select *  
    from three, four  
   where three.x=four.x;
```


Inner Joins

Display all combinations of rows with matching keys, including duplicates.

Table Three

X	A
1	a1
1	a2
2	b1
2	b2
4	d

Table Four

X	B
2	x1
2	x2
3	y
5	v

Results Set

X	A	X	B
2	b1	2	x1
2	b1	2	x2
2	b2	2	x1
2	b2	2	x2

```
select *  
    from three, four  
   where three.x=four.x;
```

Company Scenario

Display the name, designation, joining date and department name of all employees. Here is a sketch of the desired report:

EMPLOYEES REPORT			
=====			
EMPLOYEE NAME	DESIGNATION	JOINING DATE	DEPARTMENT NAME
SCOTT	ANALYST	19-APR-87	RESEARCH

CONFIDENTIAL DATA

PAGE NO.

Company Scenario

Considerations:

- **Emp** table contains employee name, job, and joining date data.
- **Dept** table contains department name.
- Both **Emp** and **Dept** contain **deptno** column name.

Inner Joins

```
set echo off
-- set page variables
set feedback on
set pagesize 25
set linesize 90
set colsep " | "
set underline "="

-- format columns
column ename format a24 heading 'EMPLOYEE NAME'
column job format a20 heading 'DESIGNATION'
column hiredate format a15 heading 'JOINING|DATE'
column dname format a20 heading 'DEPARTMENT NAME'

-- add page header and footer
tttitle center 'EMPLOYEES REPORT' skip 1 -
center '===== ' skip 2
btitle LEFT 'CONFIDENTIAL DATA' right 'PAGE NO.: ' FORMAT 99 sql.pno

-- write report to file
spool c:\notes1\emprpt.txt

select ename, job, hiredate, dname
from emp e, dept d
where e.deptno=d.deptno
order by 4,3, job;

spool off
clear columns
tttitle off
btitle off
set echo on
```

Inner Joins

SQL Output

```
SQL> ed c:\notes1\a1.sql
```

```
SQL> @c:\notes1\a1.sql
```

```
SQL> set echo off
```

EMPLOYEES REPORT

=====

EMPLOYEE NAME	DESIGNATION	JOINING DATE	DEPARTMENT NAME
=====	=====	=====	=====
CLARK	MANAGER	09-JUN-81	ACCOUNTING
KING	PRESIDENT	17-NOV-81	ACCOUNTING
MILLER	CLERK	23-JAN-82	ACCOUNTING
SMITH	CLERK	17-DEC-80	RESEARCH
JONES	MANAGER	02-APR-81	RESEARCH
FORD	ANALYST	03-DEC-81	RESEARCH
SCOTT	ANALYST	19-APR-87	RESEARCH
ADAMS	CLERK	23-MAY-87	RESEARCH
ALLEN	SALESMAN	20-FEB-81	SALES
WARD	SALESMAN	22-FEB-81	SALES
BLAKE	MANAGER	01-MAY-81	SALES
TURNER	SALESMAN	08-SEP-81	SALES
MARTIN	SALESMAN	28-SEP-81	SALES
JAMES	CLERK	03-DEC-81	SALES

CONFIDENTIAL DATA

PAGE NO.: 1

14 rows selected.

Inner Join Alternate Syntax

An inner join can also be accomplished using an alternate syntax, which limits the join to a maximum of two tables.

General form of an inner join:

```
SELECT column-1 <, ...column-n>  
FROM table-1  
INNER JOIN  
      table-2  
ON join-condition(s)  
    <other clauses>;
```



This syntax is common in SQL code.

The ON clause specifies the JOIN criteria; a WHERE clause **can** be added to subset the results.

Inner Join Alternate Syntax

```
set echo off
-- set page variables
set feedback on
set pagesize 25
set linesize 90
set colsep " | "
set underline "="

-- format columns
column ename format a24 heading 'EMPLOYEE NAME'
column job format a20 heading 'DESIGNATION'
column hiredate format a15 heading 'JOINING|DATE'
column dname format a20 heading 'DEPARTMENT NAME'

-- add page header and footer
tttitle center 'EMPLOYEES REPORT' skip 1 -
center '===== ' skip 2
bttitle LEFT 'CONFIDENTIAL DATA' right 'PAGE NO.: ' FORMAT 99 sql.pno

-- write report to file
spool c:\notes1\emprpt.txt

select ename, job, hiredate, dname
from emp e inner join dept d
on e.deptno=d.deptno
order by 4,3, job;

spool off
clear columns
tttitle off
bttitle off
set echo on
```

Poll 

Quiz

03. Multiple Choice Poll

How many tables can be combined using a single inner join?

- a. 2
- b. 32
- c. 256
- d. 512
- e. Limited only by my computer's resources
- f. No limit

03. Multiple Choice Poll – Correct Answer

How many tables can be combined using a single inner join?

- ☒ a. 2
- b. 32
- c. 256
- d. 512
- e. Limited only by my computer's resources
- f. No limit



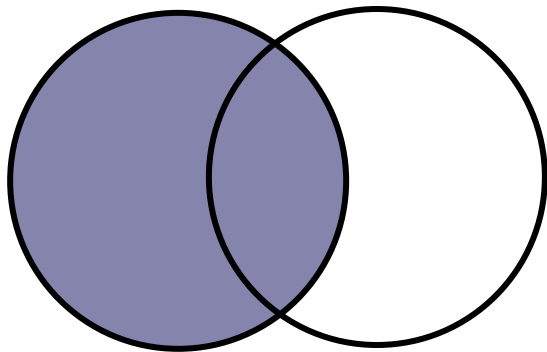
Question & Answer

Outer Joins

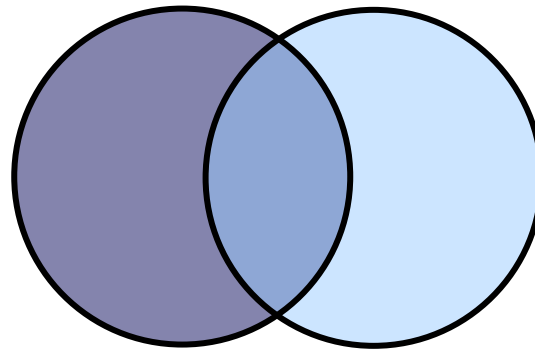
Inner joins returns only matching rows.

We can retrieve both nonmatching and matching rows using an outer join.

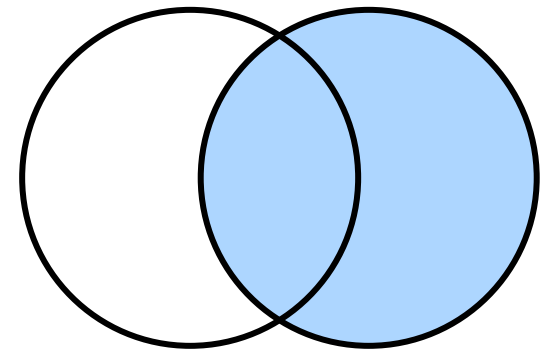
Outer joins include left, full, and right outer joins. Outer joins can process only two tables at a time.



Left



Full



Right

Compare Inner Joins And Outer Joins

The following table is a comparison of inner and outer join syntax and limitations:

Key Point	Inner Join	Outer Join
Table Limit	256	2
Join Behavior	Returns matching rows only	Returns matching and nonmatching rows
Join Options	Matching rows only	LEFT, FULL, RIGHT
Syntax changes	■ Multiple tables in the FROM clause ■ WHERE clause that specifies join criteria	ON clause that specifies join criteria

Outer Joins

Outer join syntax is similar to the inner join alternate syntax.

General form of an outer join:

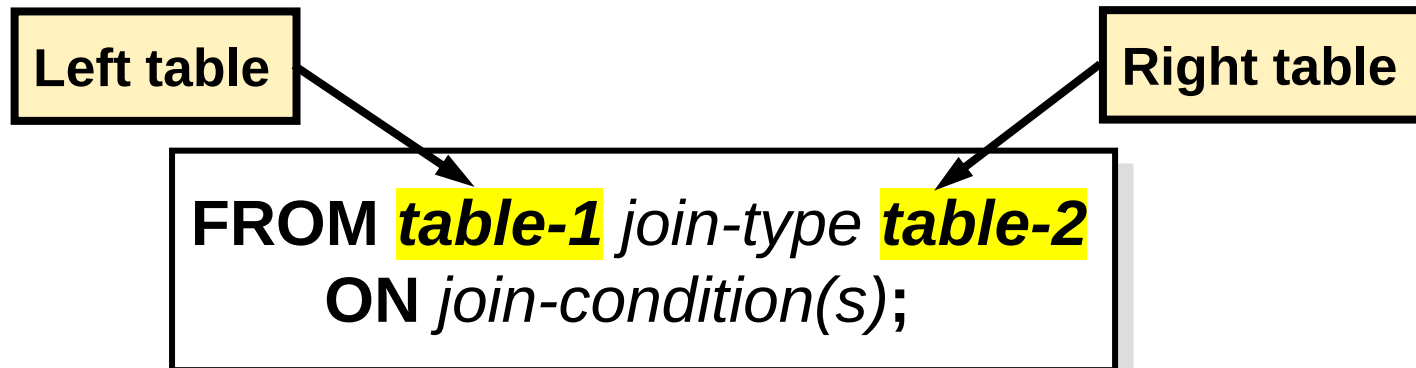
```
SELECT column-1 <, ...column-n>  
FROM table-1  
LEFT|RIGHT|FULL JOIN  
      table-2  
ON join-condition(s)  
    <other clauses>;
```

The ON clause specifies the join criteria in outer joins.

Determining Left and Right

Consider the position of the tables in the FROM clause.

- Left joins include all rows from the first (left) table, even if there are no matching rows in the second (right) table.
- Right joins include all rows from the second (right) table, even if there are no matching rows in the first (left) table.
- Full joins include all rows from both tables, even if there are no matching rows in either table.



Left Join

Table One

X	A
1	a
4	d
2	b

Table Two

X	B
2	x
3	y
5	v

```
select *  
  from one left join two  
    on one.x = two.x;
```

X	A	X	B
1	a	.	
2	b	2	x
4	d	.	

Right Join

Table Two

X	B
2	x
3	y
5	v

Table One

X	A
1	a
4	d
2	b

```
select *  
  from two right join one  
    on one.x = two.x;
```

X	B	X	A
.		1	a
2	x	2	b
.		4	d

Full Join

Table One

X	A
1	a
4	d
2	b

Table Two

X	B
2	x
3	y
5	v

```
select *  
  from one full join two  
    on one.x = two.x;
```

X	A	X	B
1	a	.	
2	b	2	x
.		3	y
4	d	.	
.		5	v

Outer Joins

```
SQL> SELECT * FROM STUDENTS;
```

NAME	AGE	SEM	MOB_NO
ANAM	16	THIRD	1111111111
MOHSIN	17	FIFTH	3333333333
JUNAID	16	THIRD	6666666666
SAAD	17	FIFTH	6767676767
HABBAN	17	THIRD	7000000000
CHAITANYA	17	THIRD	6565656565
HUZAIFA	17	THIRD	6666600000
TANZILA	17	FIFTH	6262626262

```
8 rows selected.
```

MOHSIN, SAAD,
HUZAIFA and TANZILA
have no marks details.

Consider the contents
of Students and Grades
table to perform LEFT,
RIGHT and FULL
OUTER JOINS.

```
SQL> SELECT * FROM GRADES;
```

SNAME	C_CODE	MARKS
ANAM	DMS	68
ANAM	CGR	64
JUNAID	DMS	66
JUNAID	CGR	67
HABBAN	DMS	70
HABBAN	CGR	65
CHAITANYA	DMS	60
CHAITANYA	CGR	62
CHETAN	DMS	65
CHETAN	CGR	64

```
10 rows selected.
```

Outer Joins

```
SQL> select * from emp;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

```
14 rows selected.
```

Consider the contents of EMP and DEPT tables to perform LEFT, RIGHT and FULL OUTER JOINS.

There are no employees in department no 40.

```
SQL> select * from dept;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Left Outer Join

```
SQL> SELECT * FROM STUDENTS S LEFT OUTER JOIN GRADES G ON S.NAME=G.SNAME;
```

NAME	AGE	SEM	MOB_NO	SNAME	C_CODE	MARKS
=====	=====	=====	=====	=====	=====	=====
ANAM	16	THIRD	1111111111	ANAM	DMS	68
ANAM	16	THIRD	1111111111	ANAM	CGR	64
JUNAID	16	THIRD	6666666666	JUNAID	DMS	66
JUNAID	16	THIRD	6666666666	JUNAID	CGR	67
HABBAN	17	THIRD	7000000000	HABBAN	DMS	70
HABBAN	17	THIRD	7000000000	HABBAN	CGR	65
CHAITANYA	17	THIRD	6565656565	CHAITANYA	DMS	60
CHAITANYA	17	THIRD	6565656565	CHAITANYA	CGR	62
MOHSIN	17	FIFTH	3333333333			
TANZILA	17	FIFTH	6262626262			
HUZAIFA	17	THIRD	6666600000			
SAAD	17	FIFTH	6767676767			

```
12 rows selected.
```

Left Outer Join

```
SQL> SELECT e.empno, e.ename, e.deptno, d.dname  
2 FROM emp e LEFT OUTER JOIN dept d  
3 ON (e.deptno = d.deptno);
```

EMPNO	ENAME	DEPTNO	DNAME
7934	MILLER	10	ACCOUNTING
7839	KING	10	ACCOUNTING
7782	CLARK	10	ACCOUNTING
7902	FORD	20	RESEARCH
7876	ADAMS	20	RESEARCH
7788	SCOTT	20	RESEARCH
7566	JONES	20	RESEARCH
7369	SMITH	20	RESEARCH
7900	JAMES	30	SALES
7844	TURNER	30	SALES
7698	BLAKE	30	SALES
7654	MARTIN	30	SALES
7521	WARD	30	SALES
7499	ALLEN	30	SALES

14 rows selected.

Here all the rows in emp table(LEFT) have matching values in dept table(RIGHT).

Right Outer Join

```
SQL> SELECT * FROM STUDENTS S RIGHT OUTER JOIN GRADES G ON S.NAME=G.SNAME;
```

NAME	AGE	SEM	MOB_NO	SNAME	C_CODE	MARKS
=====	=====	=====	=====	=====	=====	=====
ANAM	16	THIRD	1111111111	ANAM	CGR	64
ANAM	16	THIRD	1111111111	ANAM	DMS	68
JUNAID	16	THIRD	6666666666	JUNAID	CGR	67
JUNAID	16	THIRD	6666666666	JUNAID	DMS	66
HABBAN	17	THIRD	7000000000	HABBAN	CGR	65
HABBAN	17	THIRD	7000000000	HABBAN	DMS	70
CHAITANYA	17	THIRD	6565656565	CHAITANYA	CGR	62
CHAITANYA	17	THIRD	6565656565	CHAITANYA	DMS	60
				CHETAN	CGR	64
				CHETAN	DMS	65

10 rows selected.

Right Outer Join

```
SQL> SELECT e.empno, e.ename, e.deptno, d.dname
2  FROM    emp e RIGHT OUTER JOIN dept d
3  ON      (e.deptno = d.deptno);
```

EMPNO	ENAME	DEPTNO	DNAME
7782	CLARK	10	ACCOUNTING
7839	KING	10	ACCOUNTING
7934	MILLER	10	ACCOUNTING
7566	JONES	20	RESEARCH
7902	FORD	20	RESEARCH
7876	ADAMS	20	RESEARCH
7369	SMITH	20	RESEARCH
7788	SCOTT	20	RESEARCH
7521	WARD	30	SALES
7844	TURNER	30	SALES
7499	ALLEN	30	SALES
7900	JAMES	30	SALES
7698	BLAKE	30	SALES
7654	MARTIN	30	SALES
			OPERATIONS

15 rows selected.

Here OPERATIONS department in dept table(RIGHT) have no matching values in emp table(LEFT).

Full Outer Join

```
SQL> SELECT * FROM STUDENTS S FULL OUTER JOIN GRADES G ON S.NAME=G.SNAME;
```

NAME	AGE	SEM	MOB_NO	SNAME	C_CODE	MARKS
=====	=====	=====	=====	=====	=====	=====
ANAM	16	THIRD	1111111111	ANAM	CGR	64
ANAM	16	THIRD	1111111111	ANAM	DMS	68
MOHSIN	17	FIFTH	3333333333			
JUNAID	16	THIRD	6666666666	JUNAID	CGR	67
JUNAID	16	THIRD	6666666666	JUNAID	DMS	66
SAAD	17	FIFTH	6767676767			
HABBAN	17	THIRD	7000000000	HABBAN	CGR	65
HABBAN	17	THIRD	7000000000	HABBAN	DMS	70
CHAITANYA	17	THIRD	6565656565	CHAITANYA	CGR	62
CHAITANYA	17	THIRD	6565656565	CHAITANYA	DMS	60
HUZAIFA	17	THIRD	6666600000			
TANZILA	17	FIFTH	6262626262			
				CHETAN	CGR	64
				CHETAN	DMS	65

```
14 rows selected.
```

Right Outer Join

```
SQL> SELECT e.empno, e.ename, e.deptno, d.dname  
2 FROM emp e FULL OUTER JOIN dept d  
3 ON (e.deptno = d.deptno);
```

EMPNO	ENAME	DEPTNO	DNAME
7369	SMITH	20	RESEARCH
7499	ALLEN	30	SALES
7521	WARD	30	SALES
7566	JONES	20	RESEARCH
7654	MARTIN	30	SALES
7698	BLAKE	30	SALES
7782	CLARK	10	ACCOUNTING
7788	SCOTT	20	RESEARCH
7839	KING	10	ACCOUNTING
7844	TURNER	30	SALES
7876	ADAMS	20	RESEARCH
7900	JAMES	30	SALES
7902	FORD	20	RESEARCH
7934	MILLER	10	ACCOUNTING
			OPERATIONS

15 rows selected.

Here again OPERATIONS department in dept table(RIGHT) have no matching values in emp table(LEFT).

Qualifying Ambiguous Column Names

- Use table prefixes to qualify column names that are in multiple tables.
- Use table prefixes to improve performance.
- Use column aliases to distinguish columns that have identical names but reside in different tables.
- Do not use aliases on columns that are identified in the **USING** clause and listed elsewhere in the SQL statement.

Using a Table Alias

- An *alias* is a table nickname.
- We use table aliases to simplify queries.
- We can assign an alias to a table by following the **table name** in the FROM clause and a **nickname** for the table.
- Then we use the alias in other clauses of the QUERY statement.
- General form of the **FROM** clause for Table Alias:

```
SELECT alias-1.column-1<, ...alias-2.column-n>  
      FROM <table-1> <alias-1>  
           join-type  
           <table-2> <alias-2>  
      ON join-condition(s)  
           <other clauses>;
```

Using a Table Alias

```
SELECT e.empno, e.ename, d.dname, d.deptno
FROM emp e left join dept d
ON e.deptno=d.deptno;
```

```
SELECT e.empno, e.ename, d.dname, deptno
FROM emp e JOIN dept d
USING (deptno);
```

```
SQL> SELECT e.empno, e.ename, d.dname, d.deptno
2 FROM emp e left join dept d
3 ON e.deptno=d.deptno;
```

EMPNO	ENAME	DNAME	DEPTNO
7934	MILLER	ACCOUNTING	10
7839	KING	ACCOUNTING	10
7782	CLARK	ACCOUNTING	10
7902	FORD	RESEARCH	20
7876	ADAMS	RESEARCH	20
7788	SCOTT	RESEARCH	20
7566	JONES	RESEARCH	20
7369	SMITH	RESEARCH	20
7900	JAMES	SALES	30
7844	TURNER	SALES	30
7698	BLAKE	SALES	30
7654	MARTIN	SALES	30
7521	WARD	SALES	30
7499	ALLEN	SALES	30

69 14 rows selected.

```
SQL> SELECT e.empno, e.ename, d.dname, deptno
2 FROM emp e JOIN dept d
3 USING (deptno);
```

EMPNO	ENAME	DNAME	DEPTNO
7782	CLARK	ACCOUNTING	10
7839	KING	ACCOUNTING	10
7934	MILLER	ACCOUNTING	10
7566	JONES	RESEARCH	20
7902	FORD	RESEARCH	20
7876	ADAMS	RESEARCH	20
7369	SMITH	RESEARCH	20
7788	SCOTT	RESEARCH	20
7521	WARD	SALES	30
7844	TURNER	SALES	30
7499	ALLEN	SALES	30
7900	JAMES	SALES	30
7698	BLAKE	SALES	30
7654	MARTIN	SALES	30

14 rows selected.

Creating Joins with the USING Clause

- If several columns have the same names but the data types do not match, the **NATURAL JOIN** clause can be modified with the **USING** clause to specify the columns that should be used for an equijoin.
- Use the **USING** clause to match only one column when more than one column matches.
- Do not use a table name or alias in the referenced columns.

```
SELECT e.empno, e.ename, d.dname, deptno
FROM   emp e JOIN dept d
USING (deptno);
```

Joining Column Names

EMP

EMPNO	ENAME	DEPTNO
7782	CLARK	10
7839	KING	10
7934	MILLER	10
7566	JONES	20
7902	FORD	20
7876	ADAMS	20
7369	SMITH	20
7788	SCOTT	20
7521	WARD	30
7844	TURNER	30
7499	ALLEN	30
7900	JAMES	30
7698	BLAKE	30
7654	MARTIN	30

DEPT

DEPTNO	DNAME
10	ACCOUNTING
10	ACCOUNTING
10	ACCOUNTING
20	RESEARCH
20	RESEARCH
20	RESEARCH
20	RESEARCH
20	RESEARCH
30	SALES
30	SALES
30	SALES
30	SALES
30	SALES
30	SALES

Foreign key

Primary key

Retrieving Records with the USING Clause

```
SELECT e.empno, e.ename, d.dname, deptno  
FROM   emp e JOIN dept d  
USING (deptno);
```

EMPNO	ENAME	DNAME	DEPTNO
7782	CLARK	ACCOUNTING	10
7839	KING	ACCOUNTING	10
7934	MILLER	ACCOUNTING	10
7566	JONES	RESEARCH	20
7902	FORD	RESEARCH	20
7876	ADAMS	RESEARCH	20
7369	SMITH	RESEARCH	20
7788	SCOTT	RESEARCH	20
7521	WARD	SALES	30
7844	TURNER	SALES	30
7499	ALLEN	SALES	30
7900	JAMES	SALES	30
7698	BLAKE	SALES	30
7654	MARTIN	SALES	30

Creating Joins with the ON Clause

- The join condition for the natural join is basically an equijoin of all columns with the same name.
- Use the **ON** clause to specify arbitrary conditions or specify columns to join.
- The join condition is separated from other search conditions.
- The **ON** clause makes code easy to understand.

Retrieving Records with the ON Clause

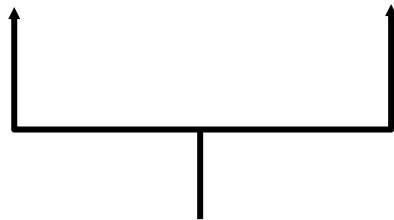
```
SELECT e.empno, e.ename, e.deptno,  
d.deptno, d.dname  
FROM emp e JOIN dept d  
ON (e.deptno=d.deptno);
```

- In this example, the **DEPTNO** columns in the **EMP** and **DEPT** table are joined using the **ON** clause.
- Wherever a **DEPTNO** in the **EMP** table equals a **DEPTNO** in the **DEPT** table, the row is returned.
- We can also use the **ON** clause to join columns that have different names.

EMPNO	ENAME	DEPTNO	DEPTNO	DNAME
7782	CLARK	10	10	ACCOUNTING
7839	KING	10	10	ACCOUNTING
7934	MILLER	10	10	ACCOUNTING
7566	JONES	20	20	RESEARCH
7902	FORD	20	20	RESEARCH
7876	ADAMS	20	20	RESEARCH
7369	SMITH	20	20	RESEARCH
7788	SCOTT	20	20	RESEARCH
7521	WARD	30	30	SALES
7844	TURNER	30	30	SALES
7499	ALLEN	30	30	SALES
7900	JAMES	30	30	SALES
7698	BLAKE	30	30	SALES
7654	MARTIN	30	30	SALES

Self-Joins(Joining a Table to Itself) Using the ON Clause

EMPNO	ENAME	MGR
7369	SMITH	7902
7499	ALLEN	7698
7521	WARD	7698
7566	JONES	7839
7654	MARTIN	7698
7698	BLAKE	7839
7782	CLARK	7839
7788	SCOTT	7566
7839	KING	
7844	TURNER	7698
7876	ADAMS	7788
7900	JAMES	7698
7902	FORD	7566
7934	MILLER	7782



MGR in the EMP table is equal to EMPNO in the SAME table.

- Sometimes we need to join a table to itself.
- To find the name of each employee's manager, we need to join the **EMP** table to itself. For example, to find the name of SCOTT's manager, we need to:
 - Find SCOTT in the **EMP** table by looking at the **ENAME** column.
 - Find the manager number for SCOTT by looking at the **MGR** column. SCOTT's manager number is 7566.
 - Find the name of the manager with **EMPNO** 7566 by looking at the **ENAME** column.
 - JONES's employee number is 7566, so JONES is SCOTT's manager.
- In this process, we look in the table twice.
- The first time we look in the table to find SCOTT in the **ENAME** column and **MGR** value of 7566.
- The second time we look in the **EMPNO** column to find 7566 and the **ENAME** column to find JONES.

Self-Joins Using the ON Clause

```
column C1 format a10 heading 'EMPLOYEE|NAME'
column C2 format a10 heading 'MANAGER|NAME'
SELECT e.ename C1, m.ename C2
FROM   emp e JOIN emp m
ON     (e.mgr = m.empno);
```

EMPLOYEE NAME	MANAGER NAME
-----	-----
FORD	JONES
SCOTT	JONES
TURNER	BLAKE
ALLEN	BLAKE
WARD	BLAKE
JAMES	BLAKE
MARTIN	BLAKE
MILLER	CLARK
ADAMS	SCOTT
BLAKE	KING
JONES	KING
CLARK	KING
SMITH	FORD

13 rows selected.

- The ON clause can also be used to join columns that have different names, within the same table or in a different table.
- The example shown is a self-join of the **EMP** table, based on the **EMPNO** and **MGR** columns.

Applying Additional Conditions to a Join

```
SELECT e.empno, e.ename, e.mgr, e.deptno,  
d.deptno, d.dname  
FROM emp e JOIN dept d  
ON (e.deptno=d.deptno)  
AND e.mgr = 7839;
```

EMPNO	ENAME	MGR	DEPTNO	DEPTNO	DNAME
7566	JONES	7839	20	20	RESEARCH
7698	BLAKE	7839	30	30	SALES
7782	CLARK	7839	10	10	ACCOUNTING

- We can apply additional conditions to the join.
- The example shown performs a join on the **EMP** and **DEPT** tables and, in addition, displays only employees who have a manager ID of 7839.
- To add additional conditions to the **ON** clause, we can add **AND** clauses. Alternatively, we can use a **WHERE** clause to apply additional conditions:

```
SELECT e.empno, e.ename, e.mgr,  
e.deptno, d.deptno, d.dname  
FROM emp e JOIN dept d  
ON (e.deptno=d.deptno)  
WHERE e.mgr = 7839;
```

Creating Three-Way Joins with the ON Clause

```
SELECT e.empno, e.ename, e.job, q.degree, d.dname
FROM   emp e
JOIN   dept d
ON     (d.deptno = e.deptno)
JOIN   qualification q
ON     e.empno = q.empno;
```

- A three-way join is a join of three tables.
- In SQL, joins are performed from left to right.
- So the first join to be performed is **EMP JOIN DEPT**.
- The first join condition can reference columns in **EMP** and **DEPT** but cannot reference columns in **QUALIFICATION**.
- The second join condition can reference columns from all three tables.

EMPNO	ENAME	JOB	DEGREE	DNAME
7369	SMITH	CLERK	B. Sc.	RESEARCH
7499	ALLEN	SALESMAN	B. Com.	SALES
7521	WARD	SALESMAN	B. Com.	SALES
7566	JONES	MANAGER	MBA	RESEARCH
7654	MARTIN	SALESMAN	M. Com.	SALES
7698	BLAKE	MANAGER	MBA	SALES
7782	CLARK	MANAGER	MBA	ACCOUNTING
7788	SCOTT	ANALYST	M. TECH.	RESEARCH
7839	KING	PRESIDENT	Doctrate	ACCOUNTING
7844	TURNER	SALESMAN	B. Com.	SALES
7876	ADAMS	CLERK	B. Com.	RESEARCH
7900	JAMES	CLERK	M. Com.	SALES
7902	FORD	ANALYST	M. TECH.	RESEARCH
7934	MILLER	CLERK	B. Com.	ACCOUNTING

Non-Equijoins

EMP

```
SQL> SELECT ENAME, SAL  
2 FROM EMP;
```

ENAME	SAL
SMITH	800
ALLEN	1600
WARD	1250
JONES	2975
MARTIN	1250
BLAKE	2850
CLARK	2450
SCOTT	3000
KING	5000
TURNER	1500
ADAMS	1100
JAMES	950
FORD	3000
MILLER	1300

14 rows selected.

SALGRADE

```
SQL> SELECT * FROM SALGRADE;
```

GRADE	LOSAL	HISAL
1	700	1200
2	1201	1400
3	1401	2000
4	2001	3000
5	3001	9999

← Salary in the **EMP** table must be between **LOSAL** and **HISAL** in the **SALGRADE** table.

- A **non-equijoin** is a join condition containing something other than an equality operator.
- The relationship between the **EMP** table and the **SALGRADE** table is an example of a non-equijoin.
- A relationship between the two tables is that the **SAL** column in the **EMP** table must be between the values in the **LOSAL** and **HISAL** columns of the **SALGRADE** table.
- The relationship is obtained using an operator other than equality (=).

Retrieving Records with Non-Equijoins

```
SELECT e.ename, e.sal, s.grade
FROM   emp e JOIN salgrade s
ON     e.sal
BETWEEN s.losal AND s.hisal;
```

ENAME	SAL	GRADE
-----	-----	-----
SMITH	800	1
JAMES	950	1
ADAMS	1100	1
WARD	1250	2
MARTIN	1250	2
MILLER	1300	2
TURNER	1500	3
ALLEN	1600	3
CLARK	2450	4
BLAKE	2850	4
JONES	2975	4
SCOTT	3000	4
FORD	3000	4
KING	5000	5

14 rows selected.

- This example creates a non-equijoin to evaluate an employee's salary grade.
- The salary must be *between* any pair of the low and high salary ranges.
- All of the employees' salaries lie within the limits that are provided by the **salgrade** table. That is, no employee earns less than the lowest value contained in the **LOSAL** column or more than the highest value contained in the **HISAL** column.

Note: Other conditions (such as $\leq$ and $\geq$) can be used, but **BETWEEN** is the simplest.

A large, stylized graphic featuring the words "Question & Answer" in a bold, sans-serif font. The word "Question" is in blue, and the ampersand "&" is in yellow. The word "Answer" is also in blue. The text is centered within a large, horizontal oval shape. The oval has a thick yellow border with a blue outline. The background is white.

Question & Answer